

No-Extra-Model Speculative Decoding for Code-Like Serving Workloads

Anonymous ACL submission

Abstract

Speculative decoding is attractive in serving systems, but many variants require a trained drafter, additional model hosting, or an adaptive controller. We study a lower-friction alternative: vLLM n-gram speculation for Qwen/Qwen3-8B, a no-extra-model method that proposes continuations from local text structure and can be enabled without training or serving a second checkpoint. Across 2,902 paired public tasks per decoding mode, n-gram speculation preserves task quality closely, with family-level quality deltas between -0.0037 and $+0.0008$. Its best-fit workload is code-like generation: on EvalPlus it is faster than greedy decoding at 1.116x latency speed ratio with a small pass@1 change, making it the more suitable option when operators need a no-extra-model speed improvement for code workloads. The same experiment shows why the method should be selected by workload rather than enabled globally: summarization, math reasoning, and instruction following preserve quality but do not show the same latency benefit. The contribution is a deployment decision rule for no-extra-model speculation: use local n-gram proposals where surface-form reuse is valuable, keep greedy or stronger speculative policies for other workloads, and instrument accepted-token counters before making mechanism-level claims.

1 Introduction

Speculative and parallel decoding methods try to reduce the sequential cost of autoregressive generation by proposing multiple tokens before committing them. The classic draft-and-verify framing shows that a verifier can preserve the target distribution while a cheaper drafter reduces the number of expensive target-model steps (Leviathan et al., 2023; Xia et al., 2023). Later systems extend this idea with self-speculation, early exits, adaptive speculation lengths, and serving policies that

change speculation behavior online (Zhang et al., 2024; Elhoushi et al., 2024; Mamou et al., 2024; Liu et al., 2025). For deployment, the most convenient variant is training-free speculation: reuse prompt or context structure, avoid a trained draft model, and keep the serving stack close to ordinary greedy decoding.

The convenience of training-free speculation creates a different research question from draft-model or self-speculative systems. The target user is not searching for the strongest possible speculative decoder after training, tuning, or adding another checkpoint; the target user wants to know whether a serving knob that reuses local text structure is worth enabling under ordinary task metrics. That regime has its own selection criterion. A method can be nearly lossless in quality yet not improve the end-to-end run once proposal construction, verification, backend scheduling, and workload shape are included. Recent work already suggests that speculative decoding is regime-dependent: adaptive policies matter, model internals matter, and serving conditions can change the value of speculation (Li et al., 2026b; Borobia et al., 2026; Li et al., 2026a; Jin et al., 2026). This paper therefore asks what a no-extra-model n-gram mode buys under a paired public benchmark matrix, not whether an unconstrained speculative system can be made fast in favorable settings.

We evaluate Qwen/Qwen3-8B (Yang et al., 2025) under vLLM (Kwon et al., 2023), comparing greedy vLLM decoding with vLLM n-gram speculation. The benchmark matrix covers CNN/DailyMail summarization, EvalPlus code generation, GSM8K mathematical reasoning, and IFEval instruction following (Hermann et al., 2015; Nallapati et al., 2016; Liu et al., 2023; Cobbe et al., 2021; Zhou et al., 2023). The full run contains 5,804 scored rows: 2,902 greedy rows and 2,902 n-gram rows, paired by task.

The result gives a direct deployment rule. N-

gram speculation preserves quality closely across all four families, and EvalPlus reaches a 1.116x latency speed ratio with a -0.0037 quality delta. In that code-like regime, the method is more suitable than greedy decoding when the operator wants a speed improvement without hosting a separate drafter. CNN/DailyMail, GSM8K, and IFEval provide the complementary selection rule: when local surface reuse is not enough to pay for proposal and verification overhead, greedy decoding or a richer speculative policy is the better operating point. The paper therefore makes a scenario-specific claim rather than a global decoder ranking.

This distinction matters for how decoding results should be read. If a paper or system report states that a speculative mode is “lossless”, the adjective can refer to several different quantities: token distribution, task quality, latency, throughput, or cost. The first two are not interchangeable with the last three. A practitioner who enables speculation in a serving backend pays for proposal construction, target-model verification, scheduler interaction, and sometimes longer generated outputs. Even when output quality remains stable, these costs can erase the apparent benefit of accepting multiple tokens per verification step. Our experiment keeps this accounting visible by reporting the task metric beside latency and token throughput for each family, and by treating speed ratios as descriptive backend outcomes rather than mechanism proof.

This scenario also changes the contribution. We do not introduce a trained drafter, a new verifier, or a tuned controller, because those choices optimize a different operating point. We instead measure the low-friction setting that requires no additional model service and no training loop. The paper-facing question is therefore not “How much can an ideal speculative-decoding method accelerate generation?” but “When does the simplest no-extra-model option cross the line from harmless quality preservation to useful serving speed?” That framing is narrower, but it is actionable for deployments that cannot add another model or tune a speculative stack.

Our contribution is a calibrated evidence package for selecting this serving mode. First, we provide a paired cross-family comparison of greedy vLLM and no-extra-model n-gram speculation on Qwen/Qwen3-8B. Second, we show that the method is a better fit for the EvalPlus code-generation slice, where it improves latency while

keeping task quality close. Third, we report task quality, latency, and tokens-per-second side by side so operators can choose the decoding mode by workload rather than by a single “lossless” label. Fourth, we identify the instrumentation needed to explain the speed advantage mechanistically: accepted-token diagnostics and verification-step counters. Figure 1 summarizes the evaluated method.

2 Related Work

Speculative decoding foundations. Speculative decoding accelerates generation by drafting candidate tokens and verifying them with the target model. Leviathan et al. (2023) formulate a lossless speculative sampling procedure for autoregressive transformers, while Xia et al. (2023) study speculative execution for sequence-to-sequence generation. These anchor papers make clear why the draft/verify abstraction is attractive: correctness can be preserved when verification is exact. They also motivate our paired quality measurement, since any practical speed result is unconvincing if it silently changes task outputs.

Self-speculation and training-free variants. Several approaches reduce or remove the need for an external draft model. Zhang et al. (2024) and Elhoushi et al. (2024) use internal model structure or early exits for self-speculation. Other methods change the speculation shape directly, including lookahead decoding (Fu et al., 2024), adaptive speculation length (Mamou et al., 2024), and more recent training-free tree or loose-verification schemes (Jin et al., 2026; Li et al., 2026a). The common pressure is to keep speculation cheap enough that accepted tokens outweigh overhead. Our n-gram setting is a strict member of this family: it uses no trained drafter and therefore depends on local repetition or prompt structure rather than learned prediction.

Training-free methods are especially sensitive to task structure. A prompt or prefix may contain repeated local patterns in code, tables, form-like instructions, or boilerplate, but the same heuristic may be much less helpful for free-form summaries or arithmetic reasoning chains. The literature often separates algorithmic possibility from deployment convenience: a method can show a clean acceptance rule while still requiring a serving backend, instrumentation, and workload mix that make the rule useful in practice. This paper sits at that inter-

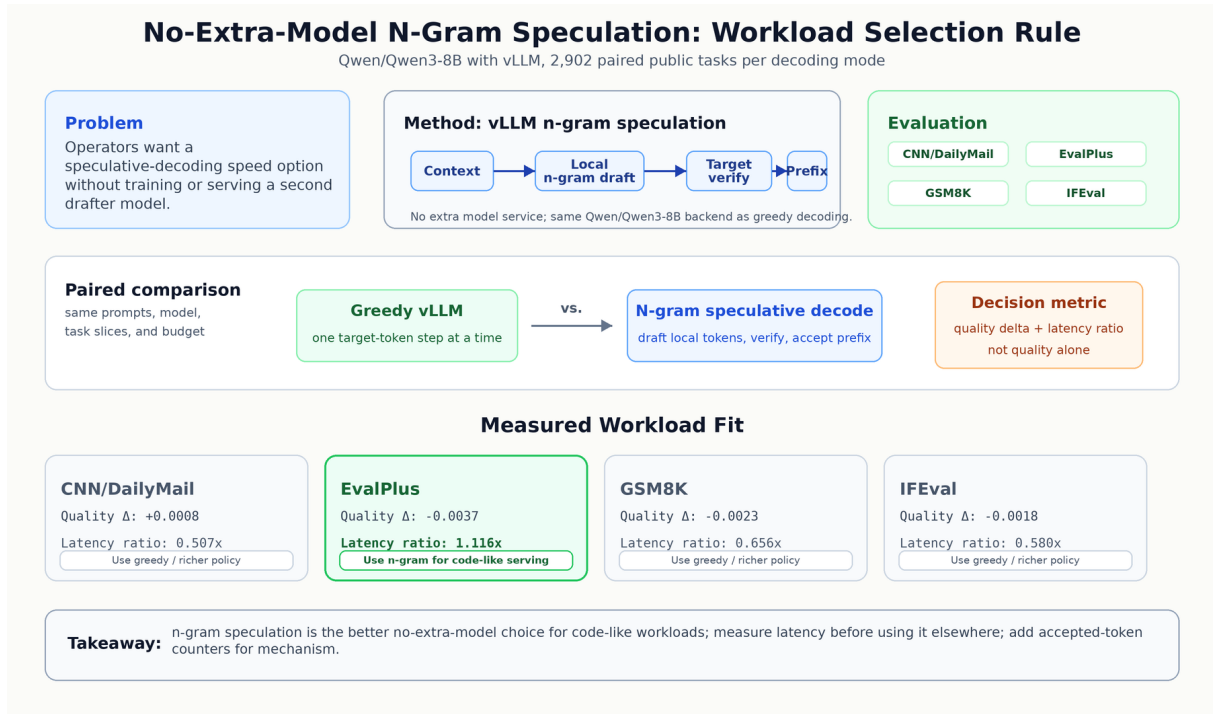


Figure 1: Overview of the evaluated no-extra-model n-gram speculation decision: under the same Qwen/Qwen3-8B vLLM backend, the paper compares greedy decoding with local n-gram speculation and selects the method by workload fit, task quality, and latency. EvalPlus is the code-like speed-positive regime.

face. We evaluate the option that is easy to enable, not the option that is easiest to make win in a controlled microbenchmark.

Adaptive and systems-aware speculation. Serving systems complicate simple speedup claims. vLLM improves serving efficiency with PagedAttention (Kwon et al., 2023), but the backend still must schedule draft construction, verification, and generation work. Speculation policies can adapt to load, model architecture, or memory pressure (Li et al., 2026b; Tiwari et al., 2025; Borobia et al., 2026). These papers motivate reporting latency and throughput as empirical outcomes rather than assuming that fewer target-model steps imply lower wall-clock time.

Systems-aware speculation also shows why workload-selection evidence is useful. Adaptive policies can shorten or lengthen draft budgets, quantize or specialize drafter computation, and change their behavior under load. Those choices may serve regimes that the simple n-gram condition does not serve, but they are not the same evaluated condition. When this paper reports that greedy remains the better latency choice on three families, the claim is about the measured vLLM n-gram mode, not about every possible speculative

decoder. The result is therefore a selection rule for a training-free default, and a motivation for future instrumentation that can explain when adaptive policies are worth their additional complexity.

Benchmark coupling of speed and quality. Decoding papers often report speed and quality together because the useful operating point is a trade-off, not a single scalar (Metel et al., 2024; Wang et al., 2025). Our benchmark suite follows that pattern across summarization, code, math, and instruction following (Nallapati et al., 2016; Liu et al., 2023; Cobbe et al., 2021; Zhou et al., 2023). The gap addressed here is not a new decoding algorithm; it is a public, cross-family measurement of a simple training-free n-gram mode whose quality and latency conclusions diverge.

The selected benchmark families deliberately stress different output regimes. CNN/DailyMail has long natural-language outputs and a summarization metric that is tolerant to paraphrase. EvalPlus has executable code outputs with local syntax and boilerplate. GSM8K combines short answers with intermediate reasoning traces. IFEval checks whether generated text obeys explicit instructions. This mix is important because a single aggregate speed number would hide the main finding: the

method’s only speed win appears on the code family, while the other families preserve quality without accelerating.

3 Method: Training-Free N-Gram Speculation Under Test

We study an evaluated decoding protocol rather than a new trained model. Both conditions serve the same Qwen/Qwen3-8B checkpoint through the same vLLM runtime. The baseline uses greedy decoding. The training-free condition enables vLLM n-gram speculation, which proposes candidate continuations from repeated n-gram structure in the prompt and generated context. Because the candidate source is local text structure, the method does not train a drafter, does not load a separate draft model, and does not add learned routing.

This design is intentionally simple. It isolates a question that is often blurred in broader speculative-decoding discussions: if a practitioner turns on an n-gram speculative mode for a small open model, how much acceleration remains once task quality is checked on real benchmarks? The answer depends on whether proposed tokens are useful enough to compensate for overhead. In tasks with repeated code structure, n-gram proposals may match future tokens often enough to help. In summarization, math, or instruction-following tasks, local repetition may be less predictive, and verification overhead can dominate.

Two-condition protocol. The paper-facing protocol has exactly two completed conditions. The first is ordinary greedy decoding in vLLM. The second enables n-gram speculation with the same checkpoint, task slice, prompt budget, and evaluation script. Each task therefore forms a paired comparison: the task prompt, benchmark family, quality metric, and generation budget are fixed, while the decoding mode changes. This pairing is the reason the analysis can compute per-family quality deltas and sensitivity filters directly from saved rows instead of comparing separate leaderboards.

What the n-gram condition can and cannot claim. The n-gram condition is training-free because it derives proposals from local text, not from a trained draft model. This makes it attractive for operators: there is no auxiliary model checkpoint to host, no additional training run, and no model-selection problem for the drafter. The same simplicity defines the selection question. N-gram spec-

ulation can exploit repeated surface structure, but it has no learned semantic model of what the target model will say next. The experiment therefore tests a useful engineering decision: whether cheap local proposals are sufficient to improve real task latency without changing task quality.

Reader-facing mechanism. At a high level, the speculative condition alternates between local proposal and target-model verification. The n-gram proposer searches for text fragments that can suggest a short continuation. vLLM then verifies candidate tokens with the target model before committing output. If enough proposals are accepted, the target model can advance more than one token for the effective cost of a verification step. If proposals are rejected or expensive to manage, the method can become slower than greedy decoding. The empirical question is therefore a balance between accepted-token benefit and proposal/verification overhead.

The run artifacts do not expose accepted-token counts, speculative-hit rates, or per-step acceptance traces. We therefore do not infer an acceptance-rate mechanism from latency alone. Instead, the method section defines the two decoding modes, and the analysis reports the mechanism gap as missing instrumentation. This distinction is important: a latency regression can be consistent with low acceptance, high overhead, backend scheduling costs, or some combination of them. The saved evidence supports the observed outcome but not a detailed acceptance-rate decomposition.

4 Experimental Setup

Benchmarks. The benchmark matrix uses four public task families, summarized in Table 1. CNN/DailyMail measures summarization quality with ROUGE-L F-measure. EvalPlus measures code generation correctness with pass@1. GSM8K measures final-answer exact match on math word problems. IFEval measures strict prompt-level instruction-following accuracy. The paired counts are 500, 542, 1,319, and 541 tasks respectively, for 2,902 tasks per decoding mode and 5,804 scored rows in the full run.

Model and runtime. All paper-facing rows use Qwen/Qwen3-8B served through vLLM. The baseline is greedy vLLM decoding. The compared training-free condition is vLLM n-gram speculation under the same task slices and model/backend.

Family	Source and task role	Pairs	Quality metric	Decoding comparison
CNN/DailyMail	public summarization test slice	500	ROUGE-L measure	F- greedy vLLM vs. vLLM n-gram speculation
EvalPlus	HumanEval+MBPP code generation	542	pass@1	same checkpoint, same prompt slice
GSM8K	grade-school math word problems	1,319	final-answer exact match	paired outputs per task id
IFEval	instruction-following prompts	541	strict prompt accuracy	same model/backend and budget

Table 1: Benchmark protocol: the study compares two decoding modes on 2,902 paired public tasks, using the family-specific quality metric and the same Qwen/Qwen3-8B vLLM backend for both methods.

This like-for-like pairing avoids comparing a speculative method on one task set with a baseline on another.

The paper does not expose local machine identifiers, private endpoints, or operator-specific orchestration details. The reproducible scientific object is the model/backend/method pairing: Qwen/Qwen3-8B, vLLM, greedy decoding, and vLLM n-gram speculation. The generated artifacts keep raw run identifiers for audit, but the manuscript discusses them only as canonical evidence, not as method components.

Metrics and pairing. For each family, we report the family-specific quality score for greedy and n-gram decoding, the n-gram-minus-greedy quality delta, a paired bootstrap interval for the quality delta, mean latency, and the latency speed ratio defined as greedy latency divided by n-gram latency. Ratios above 1.0 mean n-gram is faster. We also compute a sensitivity view that excludes pairs where either method hit the generation cap or had high repeated-line ratio. The speed statistics are descriptive because the throughput fields are produced by the batch/harness run and repeated at row granularity; we do not treat them as independent row-level speed samples.

Quality intervals and speed ratios. Quality deltas are paired by task. The bootstrap interval resamples task-level paired differences within each family, which keeps the unit of analysis aligned with the benchmark item rather than the generated token. Speed is reported more conservatively. Mean latency and output-token throughput are useful deployment signals, but in the saved artifacts they are produced by the batch/harness execution and written into rows as descriptive measurements. Consequently, the paper reports speed ratios without claiming row-independent speed significance.

Sensitivity filters. Some decoding rows can be contaminated by generation caps or high repetition. The analysis therefore recomputes quality deltas and latency ratios after excluding pairs where either output hits the generation cap or has a high repeated-line ratio. This filter removes a small number of pairs in three families and none in EvalPlus. Because the speed conclusion is unchanged after the filter, the workload-selection rule is not an artifact of a few capped or repetitive outputs.

Excluded conditions. The project attempted broader speculation coverage, but several modes did not produce paper-facing rows under the current runtime. Draft-model speculation, SGLang adaptive speculation, and suffix/lookahead dynamic speculation are therefore treated as runtime-coverage gaps rather than baselines. This is a deliberate calibration choice. A baseline without completed scored rows cannot support a comparative table, and including it as if it ran would overstate the evidence.

5 Results

Table 2 shows the central deployment result. The no-extra-model setting keeps quality close to greedy across all families, so task accuracy is not the limiting factor for this mode. The separating variable is workload shape. EvalPlus is the code-like regime where n-gram speculation is the better serving choice than greedy: it reaches a 1.116x latency ratio with a small -0.0037 pass@1 delta and requires no additional served model. CNN/DailyMail, GSM8K, and IFEval show where the same simple switch preserves quality but should not be selected for latency.

The table also shows why an aggregate statement would be misleading. Averaging the four families would mix a code-generation speed win with workloads that do not share the same surface

Benchmark	Quality metric	Pairs	Greedy	N-gram	Δ quality	Latency ratio
CNN/DailyMail	rougeL fmeasure	500	0.189	0.190	0.0008	0.507x
EvalPlus	evalplus pass at 1	542	0.697	0.694	-0.0037	1.116x
GSM8K	gsm8k final answer exact match	1319	0.848	0.846	-0.0023	0.656x
IFEval	ifeval strict prompt accuracy	541	0.815	0.813	-0.0018	0.580x

Table 2: Across 2,902 paired tasks, vLLM n-gram speculation is faster on only 1/4 benchmark families: EvalPlus reaches 1.116x latency speed ratio while dropping quality by 0.0037, and the other three families are slower than greedy decoding.

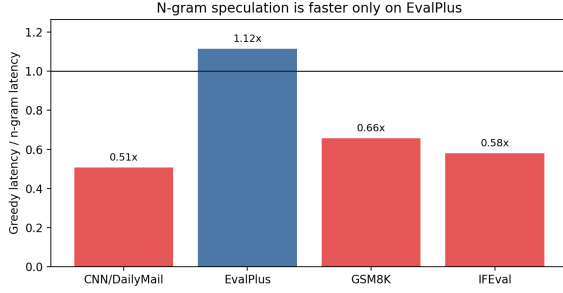


Figure 2: Latency speed ratio by benchmark family. EvalPlus is the speed-positive no-extra-model regime; CNN/DailyMail, GSM8K, and IFEval remain better served by greedy or richer speculation policies.

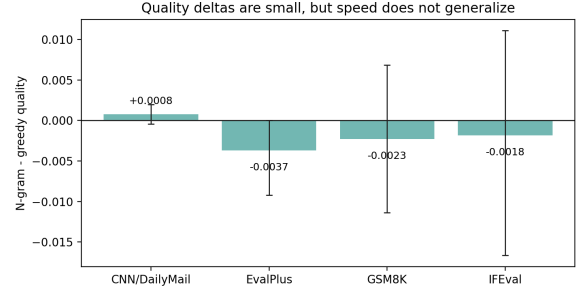


Figure 3: N-gram-minus-greedy quality deltas by benchmark family. Deltas stay within about four thousandths, supporting quality-near-lossless language but not near-lossless-speed language.

regularities. The practical decision is conditional: use the n-gram mode as a low-friction candidate for code-like generation, and treat summarization, math reasoning, and instruction following as workloads that need either stronger instrumentation or a different speculative policy.

Figure 2 makes the scenario split explicit. The measured claim is that no-extra-model speculation is a suitable speed option for code-like generation, not that it dominates richer speculative systems or every task family. CNN/DailyMail, GSM8K, and IFEval show the cost of applying a surface-form proposer outside the workloads where local repetition is likely to help. This comparison makes the method easier to deploy responsibly: choose n-gram speculation for code-like no-extra-model serving, and choose another operating point when the workload does not reward local reuse.

Figure 3 shows why a quality-only summary would be misleading. Quality deltas are small across all four families. CNN/DailyMail is slightly above zero, while EvalPlus, GSM8K, and IFEval are slightly below zero. This supports the phrase near-lossless only for task quality. It does not justify broad claims about latency.

The quality intervals reinforce the same distinction. CNN/DailyMail’s quality delta is +0.0008

with a 95% interval that crosses zero. EvalPlus drops by 0.0037 with an interval from -0.0092 to 0.0000 . GSM8K and IFEval are also slightly below zero with intervals that cross zero. These values are small enough to support cautious quality-near-lossless wording, but latency remains the deciding deployment metric in Figure 2.

Per-family interpretation. EvalPlus is the strongest fit for the method: code generation often repeats syntax, indentation, names, and local boilerplate, and the measured run converts that structure into a 1.116x latency speed ratio. CNN/DailyMail, GSM8K, and IFEval are different operating points. Their quality scores remain close to greedy, but their latency ratios favor greedy decoding under the current vLLM n-gram configuration. The comparison therefore answers the deployment question: n-gram speculation is better for the code-like no-extra-model regime, while the other families should use greedy decoding or a stronger speculative policy.

Quality is not the deployment bottleneck. Across all four families, the quality deltas occupy a narrow band from -0.0037 to $+0.0008$. The latency ratios occupy a much wider and more consequential range, from 0.507x to 1.116x. This

asymmetry is the main empirical lesson. For this no-extra-model mode, preserving task outputs is necessary but not enough; the method should be selected where it also improves latency. A deployment evaluation that stops at task quality would miss the main selection criterion.

6 Analysis and Ablation

Sensitivity to capped or repetitive generations.

The analysis excludes pairs where either method hit the generation cap or had high repeated-line ratio. This removes 9 CNN/DailyMail pairs, 4 GSM8K pairs, and 24 IFEval pairs; EvalPlus has none removed under this filter. The broad speed conclusion does not change. Sensitivity latency ratios remain 0.507x, 1.116x, 0.656x, and 0.580x for CNN/DailyMail, EvalPlus, GSM8K, and IFEval.

The sensitivity result matters because cap-hit rows can distort both quality and timing. If a method reaches the generation cap more often, its latency may increase for reasons unrelated to speculation itself. If a method repeats lines, task quality can degrade while token throughput appears superficially high. Removing these pairs is therefore a conservative check. The fact that the same family-level speed pattern remains after filtering strengthens the deployment conclusion: the code-like advantage is stable under the sensitivity view, and the other workloads remain outside the method’s best-fit region.

Why the EvalPlus advantage matters. The EvalPlus result is the positive speed case and should not be averaged away. Code generation often contains repeated local syntax, boilerplate, and structural tokens, so n-gram proposals can plausibly be more useful there. However, this remains a task-family observation rather than a full mechanism proof: the saved rows do not include accepted-token or hit-rate counters.

This is also the family where a training-free heuristic is most plausible. Code often repeats indentation, brackets, import patterns, function signatures, and local variable names. An n-gram proposer can exploit such surface regularities without understanding the program. The measured speedup is modest but useful in the intended setting because it requires no second model service. That makes n-gram speculation a practical candidate for code-serving workloads where operational simplicity is part of the objective.

Comparator scope. The project also recorded runtime-coverage gaps for off-the-shelf draft-model speculation, SGLang adaptive speculation, and suffix lookahead dynamic speculation under the current runtime. These modes have zero paper-facing rows and are therefore not included as positive baselines. Reporting them as coverage gaps keeps the comparison focused on completed evidence rather than implying that an unexecuted method was successfully compared.

The comparator scope also clarifies the paper’s contribution. The current evidence answers a completed serving question: when is greedy vLLM versus vLLM n-gram speculation the better choice under a no-extra-model constraint? It does not rank every possible speculative decoder. That distinction lets the paper make a confident claim for the code-like regime while leaving a repaired multi-method matrix as future work.

Instrumentation gap. The main missing diagnostic is accepted-token evidence. If the run had exposed per-step accepted-token counts or speculative-hit rates, the analysis could separate low proposal quality from backend overhead. Without those counters, a latency regression has several possible explanations. The n-gram proposer may offer few useful tokens; verification may dominate; scheduler interactions may offset accepted tokens; or output-length differences may affect runtime. The paper therefore reports the observed speed and quality outcomes, then marks the mechanistic explanation as under-instrumented.

Why not claim distributional losslessness. Some speculative decoding algorithms preserve the exact target distribution under a verifier. This experiment does not establish that property for the paper-facing task outputs. We measure task-level quality, not token-level distributional equivalence. The near-lossless phrase is therefore restricted to benchmark quality deltas. That calibration avoids turning a task metric into a stronger decoding-theory claim.

Result-to-decision calibration. Table 3 turns the measured rows into paper wording. This is not only a presentation device; it protects the central deployment rule from drift during drafting. The EvalPlus result supports selecting n-gram speculation for code-like no-extra-model serving. The other families support selecting greedy or a richer speculative policy when local surface reuse does not reduce latency. The quality-near-lossless claim

Decision class	Evidence observed	Paper wording	Scope guard
Global speed	Faster on 1/4 families; three latency ratios below 1.0x	n-gram speedup is workload-dependent	do not generalize the EvalPlus gain to every workload
EvalPlus speed	1.116x latency ratio with -0.0037 pass@1 delta	code-like serving is the best-fit no-extra-model regime	the result is not a universal decoder ranking
Quality preservation	quality deltas range from -0.0037 to $+0.0008$	close to lossless in task quality for this matrix	distributionally lossless decoding
Mechanism	no accepted-token or speculative-hit counters in rows/logs	observed latency and quality outcome; mechanism still needs counters	accepted-token explanation or hit-rate diagnosis

Table 3: Decision calibration from the run evidence: the full matrix supports near-lossless quality deltas, an EvalPlus speed advantage, and workload-specific deployment guidance.

is supported for task metrics in this matrix, while accepted-token mechanism explanations require additional counters.

Next instrumentation. Two additions would strengthen the method explanation. First, the serving run would need accepted-token, speculative-hit, or verification-step counters so that latency could be decomposed into proposal quality and backend overhead. Second, the unexecuted training-free modes would need repaired full rows under the same public benchmark slices. With the current evidence, the conclusion is already useful for deployment: simple n-gram speculation preserves task quality closely and is the better no-extra-model choice for the measured code-like workload.

Why the deployment rule is useful. The completed matrix changes an operational decision. For code-like workloads, no-training n-gram speculation is a credible first serving knob because it improves latency without another model service. For summarization, math, and instruction-following workloads, the same low-friction knob is not the right default under the current configuration. That result is a practical selection rule for future training-free speculation work: quality preservation is a necessary condition for deployment, but workload fit decides whether the mode saves time.

7 Deployment Guidance

Table 4 summarizes the deployment guidance that shapes the paper’s claims. The main decision variable is workload fit: code-like generation benefits from local surface reuse in this run, while the other families are better served by greedy decoding or richer speculation policies. The second decision variable is observability. Because accepted-token and speculative-hit counters are absent, the paper

Decision factor	Scope	Paper treatment
Workload fit	three benchmark families	use n-gram first for code-like workloads; use other policies elsewhere
Missing counters	all n-gram rows	do not infer acceptance rates
Draft-model gap	draft-model mode	coverage gap, not a completed comparator
SGLang gap	SGLang mode	coverage gap, not a completed comparator
Suffix gap	suffix/lookahead mode	coverage gap, not a completed comparator

Table 4: Deployment-guidance taxonomy from the run evidence: EvalPlus is the code-like speed-positive regime, accepted-token diagnostics are absent, and three alternative speculation modes remain runtime-coverage gaps rather than completed comparators.

reports the measured speed advantage and selection rule without assigning the advantage to a specific acceptance mechanism. The third decision variable is comparator scope: alternative speculation modes remain runtime-coverage gaps and cannot broaden the completed comparison.

Guidance 1: select by workload shape. The useful selection rule is not only a quality check. CNN/DailyMail, GSM8K, and IFEval all keep quality close to greedy but do not gain latency under n-gram speculation. EvalPlus does gain latency, which makes the method more suitable for code-like serving workloads under a no-extra-model constraint.

Guidance 2: instrument accepted tokens. The second rule is observability. The run records n-gram configuration and task outcomes but not accepted-token counts or speculative-hit traces. Those counters would explain why the code-like

family benefits and why the other families do not. Until then, the paper claims a measured deployment advantage, not a complete acceptance-rate mechanism.

Guidance 3: compare only completed modes.

The third rule is comparator coverage. Alternative training-free or low-training speculation paths did not yield scored rows in this run. Treating those runtime coverage gaps as missing evidence keeps the paper focused on the completed comparison instead of turning it into an unsupported ranking of all speculative methods.

Practical takeaway. The resulting rule of thumb is actionable: enable training-free n-gram speculation first for code-like workloads where local textual reuse is expected, and evaluate both quality and latency before using it elsewhere. A near-zero quality delta is not enough by itself; the method is attractive when it also improves the serving metric that motivated speculation.

8 Conclusion

No-extra-model n-gram speculation on Qwen/Qwen3-8B solves a real deployment friction: it can be tested without training or serving a separate drafter. The completed matrix shows how to use that option. On EvalPlus, the method is the better no-extra-model serving choice than greedy decoding, reaching a 1.116x latency speed ratio with a small quality change. On summarization, math reasoning, and instruction following, greedy decoding or richer speculation policies remain more suitable under the current configuration. The contribution is a closed deployment rule for a low-friction speculative mode, plus the instrumentation target needed to explain and extend the code-like advantage.

Limitations

This draft evaluates one model family, one serving runtime, and one training-free n-gram speculation mode. Speed ratios are descriptive because the throughput fields are produced by the batch/harness run rather than independent row-level timing experiments. The saved artifacts do not include accepted-token counts, speculative-hit rates, or per-step verification traces, so the paper cannot attribute speed regressions to a specific mechanism. The unexecuted modes are implementation/runtime coverage evidence, not completed comparative baselines.

The benchmark suite is public and spans four families, but it is still a bounded workload sample. It does not cover chat dialogue, retrieval-augmented generation, long-context code editing, multilingual generation, or production traffic with mixed request lengths. The measured conclusion should therefore be read as a deployment rule for this model/backend/task matrix, not as a universal law about every n-gram proposer. A future run with accepted-token counters and additional public workloads could identify a more precise operating region.

The paper also does not tune the speculative condition across a large configuration grid. This is intentional for the current study: the comparison tests a simple training-free setting against greedy decoding under matched tasks. Stronger general claims would require a preregistered search space, held-out validation of any selected setting, and a re-run of the full public matrix after selection. Without that additional evidence, we present the EvalPlus speedup as a measured no-extra-model serving advantage rather than as a tuned optimum.

Ethical Considerations

The study uses public benchmarks and focuses on decoding efficiency. The main ethical risk is overclaiming: a deployment team could waste compute or misstate environmental savings if a narrow EvalPlus speedup were presented as a broad training-free acceleration result. We therefore report the workload-selection scope directly and separate quality preservation from latency improvement.

Efficiency claims also affect environmental reporting. A decoding method that preserves quality but increases latency may raise total serving cost for a workload, even if it appears attractive in a token-level description. Reporting the three families that favor greedy alongside the one faster n-gram family reduces the risk of selective deployment narratives. The study does not involve human-subject data collection, user profiling, or private model outputs; its ethical focus is accurate measurement and cautious communication of efficiency evidence.

References

Hector Borobia, Elies Seguí-Mas, and Guillermina Tormo-Carbó. 2026. [Component-aware self-](#)

715	speculative decoding in hybrid language models.	Jonathan Mamou, Oren Pereg, Daniel Korat, Moshe	769
716	<i>Preprint</i> , arXiv:2605.01106.	Berchansky, Nadav Timor, Moshe Wasserblat, and	770
717	Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian,	Roy Schwartz. 2024. Dynamic speculation looka-	771
718	Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias	head accelerates speculative decoding of large lan-	772
719	Plappert, Jerry Tworek, Jacob Hilton, Reiichiro	guage models . <i>Preprint</i> , arXiv:2405.04304.	773
720	Nakano, Christopher Hesse, and John Schulman.		
721	2021. Training verifiers to solve math word prob-	Michael R. Metel, Peng Lu, Boxing Chen, Mehdi Reza-	774
722	lems . <i>Preprint</i> , arXiv:2110.14168.	gholizadeh, and Ivan Kobzyev. 2024. Draft on the fly:	775
723	Mostafa Elhoushi, Akshat Shrivastava, Diana Liskovich,	Adaptive self-speculative decoding using cosine sim-	776
724	Basil Hosmer, Bram Wasti, Liangzhen Lai, Anas	ilarity . In <i>Findings of the Association for Computa-</i>	777
725	Mahmoud, Bilge Acun, Saurabh Agarwal, Ahmed	<i>tional Linguistics: EMNLP 2024</i> , pages 2267–2272,	778
726	Roman, Ahmed A Aly, Beidi Chen, and Carole-	Miami, Florida, USA. Association for Computational	779
727	Jean Wu. 2024. Layerskip: Enabling early exit	Linguistics.	780
728	inference and self-speculative decoding . <i>Preprint</i> ,		
729	arXiv:2404.16710.	Ramesh Nallapati, Bowen Zhou, Cicero dos Santos,	781
730	Yichao Fu, Peter Bailis, Ion Stoica, and Hao Zhang.	Çağlar Gulçehre, and Bing Xiang. 2016. Abstrac-	782
731	2024. Break the sequential dependency of llm	tive text summarization using sequence-to-sequence	783
732	inference using lookahead decoding . <i>Preprint</i> ,	RNNs and beyond . In <i>Proceedings of the 20th</i>	784
733	arXiv:2402.02057.	<i>SIGNLL Conference on Computational Natural Lan-</i>	785
734	Karl Moritz Hermann, Tomáš Kočiský, Edward Grefen-	<i>guage Learning</i> , pages 280–290, Berlin, Germany.	786
735	stette, Lasse Espeholt, Will Kay, Mustafa Suleyman,	Association for Computational Linguistics.	787
736	and Phil Blunsom. 2015. Teaching machines to read		
737	and comprehend . <i>Preprint</i> , arXiv:1506.03340.	Rishabh Tiwari, Haocheng Xi, Aditya Tomar, Coleman	788
738	Tao Jin, Phuong Minh Nguyen, and Naoya Inoue. 2026.	Hooper, Sehoon Kim, Maxwell Horton, Mahyar Na-	789
739	Goose: Anisotropic speculation trees for training-free	jibi, Michael W. Mahoney, Kurt Keutzer, and Amir	790
740	speculative decoding . <i>Preprint</i> , arXiv:2604.02047.	Gholami. 2025. Quantspec: Self-speculative decod-	791
741	Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying	ing with hierarchical quantized kv cache . <i>Preprint</i> ,	792
742	Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E.	arXiv:2502.10424.	793
743	Gonzalez, Hao Zhang, and Ion Stoica. 2023. Ef-	Xuekang Wang, Shengyu Zhu, and Xueqi Cheng.	794
744	ficient memory management for large language	2025. Speculative safety-aware decoding . <i>Preprint</i> ,	795
745	model serving with pagedattention . <i>Preprint</i> ,	arXiv:2508.17739.	796
746	arXiv:2309.06180.	Heming Xia, Tao Ge, Peiyi Wang, Si-Qing Chen, Furu	797
747	Yaniv Leviathan, Matan Kalman, and Yossi Matias.	Wei, and Zhifang Sui. 2023. Speculative decod-	798
748	2023. Fast inference from transformers via spec-	ing: Exploiting speculative execution for accelerating	799
749	ulative decoding . <i>Preprint</i> , arXiv:2211.17192.	seq2seq generation . <i>Preprint</i> , arXiv:2203.16487.	800
750	Jinze Li, Yixing Xu, Guanchen Li, Shuo Yang, Jin-	An Yang, Anfeng Li, Baosong Yang, Beichen Zhang,	801
751	feng Xu, Xuanwu Yin, Dong Li, Edith C. H.	Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao,	802
752	Ngai, and Emad Barsoum. 2026a. Training-free	Chengen Huang, Chenxu Lv, Chujie Zheng, Day-	803
753	loosely speculative decoding: Accepting semanti-	iheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao	804
754	cally correct drafts beyond exact match . <i>Preprint</i> ,	Ge, Haoran Wei, Huan Lin, Jialong Tang, and 41	805
755	arXiv:2511.22972.	others. 2025. Qwen3 technical report . <i>Preprint</i> ,	806
756	Rui Li, Zhaoning Zhang, Libo Zhang, Huaimin Wang,	arXiv:2505.09388.	807
757	Xiang Fu, and Zhiquan Lai. 2026b. Nightjar: Dy-	Jun Zhang, Jue Wang, Huan Li, Lidan Shou, Ke Chen,	808
758	namic adaptive speculative decoding for large lan-	Gang Chen, and Sharad Mehrotra. 2024. Draft	809
759	guage models serving . <i>Preprint</i> , arXiv:2512.22420.	& verify: Lossless large language model accel-	810
760	Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and	eration via self-speculative decoding . <i>Preprint</i> ,	811
761	Lingming Zhang. 2023. Is your code generated	arXiv:2309.08168.	812
762	by chatgpt really correct? rigorous evaluation of	Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha	813
763	large language models for code generation . <i>Preprint</i> ,	Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and	814
764	arXiv:2305.01210.	Le Hou. 2023. Instruction-following evaluation for	815
765	Tianyu Liu, Yun Li, Qitan Lv, Kai Liu, Jianchen Zhu,	large language models . <i>Preprint</i> , arXiv:2311.07911.	816
766	Winston Hu, and Xiao Sun. 2025. Pearl: Paral-		
767	lel speculative decoding with adaptive draft length .	A Reproducibility Appendix	817
768	<i>Preprint</i> , arXiv:2408.11850.	Canonical run. The paper is based on the full	818
		paired Qwen/Qwen3-8B run recorded in the project	819
		artifact manifest. The canonical paper-facing de-	820
		rived artifacts are the main results table, the full	821
		matrix, the deployment-guidance taxonomy, and	822
		the result-to-claim mapping generated during the	823
		analysis stage.	824

Regeneration. The analysis report records the exact regeneration command and run identifier. The LaTeX table includes in this draft were generated from the canonical TSV artifacts by the draft table generator.

Figure provenance. The speed-ratio and quality-delta data figures have provenance sidecars in the project figure directory. Conceptual Figure 1 is generated deterministically by the figure-building script in the project figure directory from the paper-facing result values and saved as the workload-decision PNG used in Figure 1.

Claim calibration. The paper claims workload-specific deployment value rather than broad speedup or near-lossless-speed behavior. Near-lossless quality is supported by small family-level quality deltas. EvalPlus speedup is reported as the measured code-like no-extra-model advantage because it is the family where n-gram speculation improves latency while keeping quality close.